

Mutation Testing Report

I used `pytest-gremlins` as the mutation tool.

RomanConverter results

The RomanConverter test suite only passes for `adendulu1`. Many other implementations either fail behavior checks, omit `from_roman`, or implement a different validation contract. So I used `adendulu1`.

Test suite	Result
Zhuoyang	41/41 killed, 100%
adendulu	41/41 killed, 100%
dracup	41/41 killed, 100%
heowe from_roman	41/41 killed, 100%
heowe to_roman	41/41 killed, 100%
ross from_roman	41/41 killed, 100%
ross to_roman	41/41 killed, 100%
wang	41/41 killed, 100%

Since it has already achieved 100%, so no augmentation was needed for that chosen suite.

Operator	Killed	Survived
comparison	19	0
boundary	8	0
boolean	7	0
arithmetic	5	0
return	2	0

TCAS results

Initial original tests, using full test selection:

- 83 mutants total
- 41 killed
- 42 survived
- mutation score: 49.4%

Operator	Killed	Survived
comparison	14	16

Operator	Killed	Survived
boundary	10	10
boolean	9	9
return	8	6
arithmetic	0	1

The hardest mutants were comparison, boundary, and boolean mutants around TCAS decision boundaries: strict altitude comparisons, threshold-layer selection, 300 and 600 boundaries, alim comparisons, and compound boolean expressions in the top-level condition. The lone arithmetic survivor was the climb-inhibit + 100 bias being changed to subtraction.

After adding mutation-focused tests:

- 83 mutants total
- 83 killed
- 0 survived
- mutation score: 100%

Augmented operator breakdown:

Operator	Killed	Survived
comparison	30	0
boundary	20	0
boolean	18	0
return	14	0
arithmetic	1	0

The new tests directly assert helper behavior and full advisory outcomes at boundary values. This killed mutants that the original predicate/clause tests missed because those tests often verified trace values or predicate toggles rather than externally visible TCAS behavior.

100% and equivalent mutants

For RomanConverter, I reached 100% mutation score. Therefore there are no remaining unkilld mutants in those final runs.

For the original TCAS suite, I reached 49.6%. But I reached 100% after addeing new tests.

Which test suite performed better?

For RomanConverter on adendu1, all suites tied at 100%, so none performed better.

For TCAS, the new suite performed better than the original suite. It improved from 49.4% to 100% because it directly checks helper behavior, strict boundaries, threshold values, climb-inhibit arithmetic, and final advisory outputs.

Commonalities between reports

Both projects show that useful mutants tend to occur at small decision points: comparisons, boundaries, boolean logic, and arithmetic used in control decisions. The reports also show that a passing baseline matters: mutation scores are meaningful only after the unmutated implementation satisfies the test contract.

Mutation operators that were hard to kill

In the original TCAS run, the hardest mutation operators were:

- **comparison**: examples include `<` to `<=`, `>` to `>=`, and `==` to `!=`.
- **boundary**: examples include shifting threshold checks around `300`, `600`, and `alim`.
- **boolean**: examples include `and` to `or` and `not x` to `x`.
- **arithmetic**: the climb-inhibit `+ 100` changed to subtraction survived in the original TCAS suite.

These were hard because TCAS depends heavily on threshold and decision logic, and the original tests did not always exercise equality and just-above/just-below cases.

Mutants that were always killed

For RomanConverter on `adendulu1`, all generated mutants were killed: comparison, boundary, boolean, arithmetic, and return mutants all had 0 survivors.

For new TCAS, all generated mutants were also killed. In the original TCAS run, no operator category was always killed.

Operators most effective at creating difficult mutants

The most effective difficult-mutant operators were comparison and boundary operators, especially around TCAS thresholds. Boolean operators were also effective in compound conditions because changing `and/or` can preserve many test outcomes unless tests assert the final advisory result under carefully chosen states.